

```

temperature | numeric(8,6)          | [...]

Partition key: RANGE (insert_time)
Indexes:
    "basilea_partitioned_pkey" PRIMARY KEY, btree (id, insert_time)
Partitions: basilea_partitioned_1950 FOR VALUES FROM ('1949-12-31
23:00:00+00') TO ('1950-12-31 23:00:00+00'),
            basilea_partitioned_1951 FOR VALUES FROM ('1950-12-31
23:00:00+00') TO ('1951-12-31 23:00:00+00'),
[....]
            basilea_partitioned_2023 FOR VALUES FROM ('2022-12-31
23:00:00+00') TO ('2023-12-31 23:00:00+00'),
            basilea_partitioned_default DEFAULT

```

The table is partitioned by year from 1950 to 2022 and there is also a default table (as explained in the Default Partition section).

The data is divided equally between all the child tables, and if we tried to execute the same query we would get the following:

```

world_temperatures=# explain analyze select extract (year from insert_time) as
year, avg(temperature) avg_temp  from basilea_partitioned group by 1 order by
2 desc limit 5;

QUERY PLAN

-----
-----
-----

Limit  (cost=13183.59..13183.61 rows=5 width=64) (actual
time=169.996..174.092 rows=5 loops=1)
  -> Sort  (cost=13183.59..13184.09 rows=200 width=64) (actual
time=169.995..174.090 rows=5 loops=1)
    Sort Key: (avg(basilea_partitioned.temperature)) DESC
    Sort Method: top-N heapsort  Memory: 25kB
  -> Finalize GroupAggregate  (cost=13127.60..13180.27 rows=200
width=64) (actual time=169.812..174.060 rows=73 loops=1)
    Group Key: (EXTRACT(year FROM basilea_partitioned.insert_time))

```

```

-> Gather Merge (cost=13127.60..13174.27 rows=400
width=64) (actual time=169.802..173.941 rows=132 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    -> Sort (cost=12127.58..12128.08 rows=200 width=64)
(actual time=140.386..140.401 rows=44 loops=3)
        Sort Key: (EXTRACT(year FROM basilea_
partitioned.insert_time))
        Sort Method: quicksort Memory: 30kB
        Worker 0: Sort Method: quicksort Memory: 29kB
        Worker 1: Sort Method: quicksort Memory: 29kB
    -> Partial HashAggregate
(cost=12116.93..12119.93 rows=200 width=64) (actual time=140.330..140.363
rows=44 loops=3)
        Group Key: (EXTRACT(year FROM basilea_
partitioned.insert_time))
        Batches: 1 Memory Usage: 64kB
        Worker 0: Batches: 1 Memory Usage: 48kB
        Worker 1: Batches: 1 Memory Usage: 64kB
    -> Parallel Append (cost=0.00..10780.64
rows=267259 width=40) (actual time=0.010..79.251 rows=213304 loops=3)
        -> Parallel Seq Scan on basilea_
partitioned_1952 basilea_partitioned_3 (cost=0.00..129.59 rows=5167
width=40) (actual time=0.010..3.176 rows=8784 loops=1)
[....]
        -> Parallel Seq Scan on basilea_
partitioned_2018 basilea_partitioned_69 (cost=0.00..128.41 rows=5153
width=40) (actual time=0.004..2.671 rows=8760 loops=1)
        -> Parallel Seq Scan on basilea_
partitioned_2021 basilea_partitioned_72 (cost=0.00..128.41 rows=5153
width=40) (actual time=0.003..2.651 rows=8760 loops=1)
        -> Parallel Seq Scan on basilea_
partitioned_default basilea_partitioned_75 (cost=0.00..21.10 rows=888
width=40) (actual time=0.000..0.000 rows=0 loops=1)
        -> Parallel Seq Scan on basilea_
partitioned_2023 basilea_partitioned_74 (cost=0.00..1.01 rows=1 width=40)
(actual time=0.008..0.009 rows=1 loops=1)
Planning Time: 0.698 ms

```

```
Execution Time: 174.250 ms
(97 rows)
```

As we can see, PostgreSQL first performs a parallel sequential scan and then a parallel append to merge all the data that has been taken from the child tables.

Now let's try to perform the same operation, but filtering for the years ranging from 2021 to 2022. On the non-partitioned table we will have the following:

```
world_temperatures=# explain analyze select extract (year from insert_
time) as year, avg(temperature) avg_temp from basilea where insert_time
>='2021-01-01' and insert_time < '2023-01-01' group by 1 order by 2 desc
limit 5;
```

QUERY PLAN

```
-----
Limit (cost=11498.05..11498.06 rows=5 width=64) (actual
time=24.532..28.875 rows=2 loops=1)
  -> Sort (cost=11498.05..11544.43 rows=18554 width=64) (actual
time=24.529..28.871 rows=2 loops=1)
    Sort Key: (avg(temperature)) DESC
    Sort Method: quicksort Memory: 25kB
    -> Finalize HashAggregate (cost=10911.56..11189.87 rows=18554
width=64) (actual time=24.467..28.859 rows=2 loops=1)
      Group Key: (EXTRACT(year FROM insert_time))
      Batches: 1 Memory Usage: 793kB
      -> Gather (cost=9133.43..10795.60 rows=15462 width=64)
(actual time=24.159..28.707 rows=6 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        -> Partial HashAggregate (cost=8133.43..8249.40
rows=7731 width=64) (actual time=20.170..20.211 rows=2 loops=3)
          Group Key: EXTRACT(year FROM insert_time)
          Batches: 1 Memory Usage: 409kB
          Worker 0: Batches: 1 Memory Usage: 409kB
          Worker 1: Batches: 1 Memory Usage: 409kB
          -> Parallel Seq Scan on basilea
(cost=0.00..8094.78 rows=7731 width=40) (actual time=16.211..18.330
rows=5840 loops=3)
```

```

Filter: ((insert_time >= '2021-01-01
00:00:00+00'::timestamp with time zone) AND (insert_time < '2023-01-01
00:00:00+00'::timestamp with time zone))

Rows Removed by Filter: 207464

Planning Time: 0.248 ms
Execution Time: 30.043 ms
(20 rows)

```

And for the partitioned table, we will have the following:

```

world_temperatures=# explain analyze select extract (year from insert_
time) as year, avg(temperature) avg_temp from basilea_partitioned where
insert_time >='2021-01-01' and insert_time < '2023-01-01' group by 1 order
by 2 desc limit 5;

QUERY PLAN

-----
Limit (cost=618.10..618.11 rows=5 width=64) (actual time=15.205..15.208
rows=2 loops=1)
  -> Sort (cost=618.10..618.60 rows=200 width=64) (actual
time=15.203..15.205 rows=2 loops=1)
    Sort Key: (avg(basilea_partitioned.temperature)) DESC
    Sort Method: quicksort Memory: 25kB
    -> HashAggregate (cost=611.78..614.78 rows=200 width=64)
(actual time=15.190..15.194 rows=2 loops=1)
      Group Key: (EXTRACT(year FROM basilea_partitioned.insert_
time))
      Batches: 1 Memory Usage: 40kB
      -> Append (cost=0.00..524.19 rows=17517 width=40) (actual
time=0.032..8.804 rows=17520 loops=1)
        -> Seq Scan on basilea_partitioned_2021
basilea_partitioned_1 (cost=0.00..217.30 rows=8758 width=40) (actual
time=0.030..4.466 rows=8759 loops=1)
          Filter: ((insert_time >= '2021-01-01
00:00:00+00'::timestamp with time zone) AND (insert_time < '2023-01-01
00:00:00+00'::timestamp with time zone))
          Rows Removed by Filter: 1

```

```

-> Seq Scan on basilea_partitioned_2022
basilea_partitioned_2 (cost=0.00..218.30 rows=8758 width=40) (actual
time=0.003..2.930 rows=8760 loops=1)
    Filter: ((insert_time >= '2021-01-01
00:00:00+00'::timestamp with time zone) AND (insert_time < '2023-01-01
00:00:00+00'::timestamp with time zone))
-> Seq Scan on basilea_partitioned_2023 basilea_
partitioned_3 (cost=0.00..1.02 rows=1 width=40) (actual time=0.006..0.006
rows=1 loops=1)
    Filter: ((insert_time >= '2021-01-01
00:00:00+00'::timestamp with time zone) AND (insert_time < '2023-01-01
00:00:00+00'::timestamp with time zone))
Planning Time: 0.439 ms
Execution Time: 15.311 ms
(17 rows)

```

The first thing we see is that PostgreSQL examines fewer child tables when there is a where clause on the field used in the partitioning; the `constraint_exclusion postgresql.conf` parameter makes this possible:

```

world_temperatures=# select * from pg_settings where name = 'constraint_
exclusion';
-[ RECORD 1 ]-
name          | constraint_exclusion
setting       | partition
unit          |

category      | Query Tuning / Other Planner Options
short_desc    | Enables the planner to use constraints to optimize
queries.
extra_desc    | Table scans will be skipped if their constraints
guarantee that no rows match the query.
context       | user
vartype       | enum
source        | default
min_val       |
max_val       |

```

```
enumvals      | {partition,on,off}  
boot_val      | partition  
reset_val     | partition  
sourcefile    |  
  
sourceline    |  
pending_restart | f
```

This parameter makes it possible for the query optimizer to exclude some child tables from the search. As you can see in the preceding code, the possible values for the `constraint_exclusion` parameter are the following:

- `on`: With this value set, PostgreSQL examines all tables.
- `off`: With this value set, PostgreSQL doesn't examine any constraints.
- `partition`: With this value, PostgreSQL checks the constraints for the `UNION ALL` subqueries and only for inheritance child tables. `partition` is the default setting.

For further information, see <https://www.postgresql.org/docs/current/runtime-config-query.html#GUC-CONSTRAINT-EXCLUSION>.

Summary

In this chapter, we introduced the topic of table partitioning in PostgreSQL. Partitioning tables is useful as they become bigger and bigger, making queries slower and slower. We started by introducing the basic concepts of partitioning. We talked about range partitioning, list partitioning, and hash partitioning. We also went through some examples of list partitioning and range partitioning using tablespaces.

We will return to talking about partitioning in *Chapter 13, Indexes and Performance Optimization*. In the next chapter, we will talk about how PostgreSQL manages users, roles, and in general, the security of our database.

Verify your knowledge

- Is it possible to perform declarative partitioning in PostgreSQL?
Yes, starting from PostgreSQL 10, it is possible to use declarative partitioning.
See the *Exploring declarative partitioning* section for more details.

- If we have a table such as:

```
CREATE TABLE mytable (
    pk serial NOT NULL,
    create_date date not null default now()::date,
    primary key (pk)
)
```

- Is it possible to partition that table for range? Yes, it is possible by writing something like the following:

```
CREATE TABLE mytable (
    pk serial NOT NULL,
    create_date date not null default now()::date,
    primary key (pk, creat_edate)
)PARTITION BY RANGE (create_date);
CREATE TABLE

forumdb=> CREATE TABLE part_tags_date_01 PARTITION OF part_tags FOR
VALUES FROM ('2023-01-01') TO ('2023-06-31');
forumdb=> CREATE TABLE part_tags_date_01 PARTITION OF part_tags FOR
VALUES FROM ('2023-07-01') TO ('2023-12-31');
```

See the *Exploring declarative partitioning* section for more details.

- What is the default partition for?

The default partition ensures that no data is lost; if PostgreSQL does not find any child table in which to store the record, then the record is saved in the default partition.

See the *The default partition* section for more details.

- Can we split the data on different disks?

Yes, we can, using tablespaces.

See the *Partitioning and tablespaces* section for more details.

- Does PostgreSQL manage indexes on partitioned tables?

Yes, PostgreSQL manages indexes on partitioned tables; if we build an index on a parent table, PostgreSQL will take care to build the index itself on all child tables.

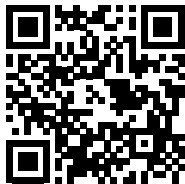
References

- PostgreSQL official documentation about table partitioning: <https://www.postgresql.org/docs/current/ddl-partitioning.html>
- PostgreSQL official documentation about inheritance: <https://www.postgresql.org/docs/current/tutorial-inheritance.html>
- PostgreSQL tuning: <https://pgtune.leopard.in.ua>
- PostgreSQL official documentation about CONSTRAINT EXCLUSION: <https://www.postgresql.org/docs/current/runtime-config-query.html#GUC-CONSTRAINT-EXCLUSION>
- PostgreSQL official documentation about trigrams: <https://www.postgresql.org/docs/current/pgtrgm.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



10

Users, Roles, and Database Security

PostgreSQL is a rock-solid database, and it pays great attention to security, providing a very rich infrastructure for handling permissions, privileges, and security policies. This chapter builds on the basic concepts introduced in *Chapter 3, Managing Users and Connections*, revisiting the role concept and extending knowledge with a particular focus on security and privileges granted to *roles* (a role can be both a user and a group of users). You will learn how to configure every aspect of a role to carefully manage security, from connection to accessing the data within a database.

PostgreSQL also provides a strong mechanism known as **Row-Level Security (RLS)**, which allows a fine-grain definition of policies to mask out part of the data to certain users.

In this chapter, you will also learn about the **Access Control List (ACL)** and the way PostgreSQL handles permissions internally, which is the result of granting or revoking privileges. Finally, you will look briefly at the password encryption algorithms that PostgreSQL provides for storing role passwords safely.

This chapter covers the following topics:

- Understanding roles
- ACLs
- Granting and revoking permissions
- RLS
- Role password encryption
- SSL connections